Generative Specification — A Field Guide

How to give an AI agent the discipline it was already trained on — but won't apply unless you make it.

Author: Juan Carlos Ghiringhelli (PragmaWorks)
Derived from: the Compendium (the long reference document, linked at the end), v5.0 (September 2026), with its corrections of 2026-10-14, the canonical source
Last synced: 2026-10-20
Status: Short practitioner summary, derived. The evidence behind it is exploratory: the studies are small and were run by the author.
License: CC BY 4.0. Reuse with attribution.

> **The arc in a paragraph.** Generative Specification (GS) began as a way to make the investment viable: write the specification, derive the rest, and verify it without a person writing or reading every line. To do that, what matters was moved out of people's heads and out of the model's context into files and checks: the specification, the document cascade, the sentinel, the gates, and the lock that ties each artifact to the specification version it came from. That substrate produced, without being designed, two forms of governance: continuity, so that what was decided and why survives who did it and who left, and state, so that where each project stands against its requirements, the rubric and the business metrics are meant to be checkable at any snapshot. We suspect, and have not measured, that this lowers the dependence of average quality on individual talent, while the exceptional keep feeding the substrate with rules and gates. That is a thesis, not a result.

---

1. The theory

This section is the author's explanation of why the method should work. It is a theory. Section 3 says how far the evidence goes.

The reader is stateless

An AI agent can generate an entire system in one session. To do it, it silently resolves a thousand implicit decisions — units, field ordering, layer ownership, error semantics — drawing on how such systems are *usually* built. Each choice is locally reasonable; across sessions, teams, and services they diverge. This is **architectural drift**.

The cause is not a weak model. It is that the model is a **stateless reader**: it begins every session with no memory of prior decisions, no shared context, and no ability to ask. Everything you did not write down is, to it, absent. So the design target is precise: **a specification from which a reader with no context can derive the correct output, alone.** That property — *derivability by a stateless reader* — is the whole discipline.

Why specifying should work: the bridge, and its asymmetry

Every structural discipline you already use — naming, SOLID, a schema, a domain language, a contract — is a **bridge between human meaning and executable code**. It carries intent across the gap. Those bridges were built for the next *human* reader.

Skilled programmers were always fluent on both banks — that fluency *is* the craft. The transformer is the first **tireless machine executor** with it, at scale: it reads intent encoded as structure and emits code that encodes intent, in both directions. And, like us, it grasps an explanation far more reliably than it reconstructs behavior from raw code.

And the bridge is **asymmetric** — this is the leverage. Training is overwhelmingly natural language; code is a small, exact, unforgiving slice where one wrong token breaks everything. The model is therefore *far stronger on human meaning than on exact code*. Encoding intent as structure — names, contracts, constraints in human-conceptual terms — **routes the hard half of the problem through the model's strong half.** That is the mechanism: specifying moves the load to the bank the model is fluent on.

Restriction is activation — take your AI to school

The model already went to school. Its training contains the entire formal tradition — Hoare logic, type theory, design by contract, SOLID, DDD, REST, deontic constraints. **It does not lack the knowledge; it lacks the instruction to apply it.**

So every constraint you name in a specification is meant to work as **activation**, not teaching. Naming "hexagonal architecture" or "idempotent handler" or "per-tenant isolation" is meant to rule out the wrong programs and select the correct one the model already knew how to write. A discipline of removal, in Robert Martin's sense: you do not add capability, you *delete the freedom to be wrong*. What remains is the program that was always latent in the model's training.

Generative Specification is an attempt to make a model that has been to school actually do the coursework.

Phase collapse

Planning, implementation, testing, review and deploy used to be separate phases. When a complete specification holds the intent, the executor pays little cost to switch tasks, and gates check the result automatically, they can run inside one working session. How much time this saves has not been measured. The risk is concrete: when one agent writes both the test and the code, "write the failing test first" can be followed in wording and broken in practice, so a gate has to enforce it.

What endures

This is the working hypothesis, not yet tested: two things should not depend on how strong the model is. The first is a check decided outside the model. A language model should not be the only verifier of its own output, so the check that says a system is sound would be a gate that is not itself a model: types, tests, mutation testing and security gates that a deterministic tool decides. The second is a record kept

outside the session: the specification and the decision trail, from which the code could be regenerated, so the intent can survive when the context closes, the author leaves, or next quarter's model is different. This is the position the work argues for. The evidence reported here tests parts of the mechanism and none of the durability claim.

---

2. What to do

One door: the sentinel navigational tree

Give the agent a single entry point that declares scope and routes to the slice each task needs — not forty files to guess among. A well-formed sentinel (a `CLAUDE.md`, `AGENTS.md` or the equivalent file your tool reads) carries five things:

```
# CLAUDE.md (the sentinel — the one door)

## Identity        What this system is (screaming architecture — the name states the domain)
## Standards       The disciplines in force (SOLID, TDD, hexagonal, conventional commits)
## Constraints     Inviolable rules and forbidden patterns (each tied to a real past
incident)
## Tool sequencing WHEN to prefer which tool — not just a list          ← most often missing
## Routing         Where each concern lives → links to the scoped child specs
```

Tool sequencing is the most common gap in our experience (no count kept): a tree that lists tools but never says *when* to prefer one forces the agent to guess — and guessing is where drift enters.

Keep the door small. The agent re-reads the sentinel every turn, and attention degrades with depth: we have seen rules near the bottom of a long file become effectively invisible late in a long session (an observation, not a measurement), and some tools cut long files off silently (~300 lines is a ceiling we use, a rule of thumb from the author's practice, not measured, and it depends on the tool). Hold `CLAUDE.md` near 250–300 lines: *name* the disciplines (SOLID, TDD), don't tutorialize them — the model already went to school. When a *spec* outgrows ~500 lines, don't load it whole; build a ~50-line **spec-map** that points each task to the exact line ranges it needs, so the agent loads one slice instead of the whole spec. We have not measured the saving.

Two logs the door must carry: each is meant to turn a one-time correction into a standing rule: - **Corrections Log** — when you tell the agent *"don't do that"* about a pattern it produced, it appends a dated one-liner (`[2026-03-12] — handle invalid cases with early-return guards, never nested conditionals`). It reads the log next session, which should make the same deviation less likely (not measured). - **Known Pitfalls** — technology traps, not behavioral ones: the flag that silently no-ops, the library whose types lie about runtime. Three parts each — what goes wrong, the wrong pattern, the right one.

The stateless reader can't remember it hit the trap yesterday; these are how the repo remembers *for* it — the ratchet (a rule set that only grows, explained below), pointed at the sentinel.

Two decision layers the tree routes to: ADR and EDR

The sentinel routes to specs, but a spec answers *what* to build, not *why it is shaped this way* — and the stateless reader needs the *why*, or on its next visit it will "improve" a deliberate constraint into a bug. Two kinds of decision record carry that rationale, and keeping them distinct is what makes the tree legible instead of a single undifferentiated pile:

- **ADR — Architecture Decision Record.** The design envelope: the cross-cutting choices that govern *how the system is built* — the language and stack, the structural style, which pattern resolves which class of problem, the dependency direction. It answers "why is it shaped this way," and reading it wrong is what makes an agent reach for the wrong pattern. This is the layer that needs an engineer who knows *how to implement* the decision, not just name it.
- **EDR — Engineering Decision Record.** The implementation layer, in two halves that must travel together: one states, *functionally*, what a unit does; the other, *how* it is implemented. Bound side by side, small, they hand the agent the full local context for a change without dragging in the whole system.

The two complement each other: the ADR gives the global design frame, the EDR the local build detail. Keep each record **small and single-purpose** (the Bounded discipline applied to decisions themselves) so the relevant slice fits the reader's window. These records are also exactly what the **Auditable** property scores — the retrievable *why* — and what makes value attribution possible later: a decision you can trace is a decision you can tie an outcome to.

How to build the sentinel, in order

Don't author the whole tree up front. Grow it in the order a stateless reader needs it:

1. **Root** `CLAUDE.md` — the one door, the five categories above, near 250–300 lines. Identity and Routing first; the rest accretes.
2. **A first spec** for the initial slice, with its acceptance criteria decidable (prescriptive, not descriptive).
3. **ADR-000** — the founding architecture decisions (stack, structural style, dependency direction), even if short. This is the frame every later generation reads against.
4. **EDRs as units land** — each meaningful implementation choice gets its two-part record, linked from the spec and tagged with the files it governs (the doc-to-code index).
5. **Split when a node outgrows its window** — a spec past ~500 lines becomes a spec-map; an architecture file past its bound gets its own child sentinel (recursive Bounded). The root always points to the children.

The test that it is working: hand a cold agent only the slice the tree routes it to, and it completes the task without reaching outside that slice. If it has to scan the whole repo, the tree is not yet doing its job.

Grade the spec the way an AI reads it: SAVED, and the layer beneath

Grade the spec as a **report card**: a letter per property from A (best) to F (worst), evidence on the surface, not an invented number. Five properties are the ones anyone accountable for what ships can read; they spell **SAVED**. Two more, Bounded and Composable, are the engineering layer that keeps

SAVED true. That makes seven to grade today. The fuller ten-property standard, the **decagon**, adds Observable, Reproducible and Evolvable. Run the seven on any repo today:

Group	Property	Ask of the spec	Removes
S — SAVED	**Self-describing**	Does the system explain itself from its surface, without its author?	Hidden intent the reader must infer
A — SAVED	**Auditable**	Is the *why* recorded and retrievable (ADRs, EDRs, commits)?	Lost rationale
V — SAVED	**Verifiable**	Is correctness *computed* by gates, not claimed ("it compiled")?	Unchecked correctness
E — SAVED	**Executable**	Are contracts run against a live system, not assumed?	Specs never tested against reality
D — SAVED	**Defended**	Are rules *enforced* (they fail the build), not advisory?	Rules the model treats as optional
eng.	**Bounded**	Can a task load only its slice, not the whole system?	An unbounded surface nobody can scan
eng.	**Composable**	Can a unit be understood and changed in isolation?	Tangled coupling

In the theory, **Bounded** is the keystone (its one direct test, RND-1 in Section 3, showed no effect at that scale): a bounded, self-describing spec activates the model's *relevant* knowledge instead of its full prior. The five SAVED properties are what an auditor or non-engineer reads first; the engineering layer (and the three further properties that complete the decagon) are how engineers keep it sound. The scorecard reports a letter per property, never a bare score.

Maturity levels, as a reference scale. Separate from the letters, a project can be placed on a five-step reference scale, L1 to L5, in the same spirit as the level scales other maturity models use; no equivalence with any of them is claimed. It is a scale to read against, not a result: this guide places no project on it. The free audit returns a letter per property, not a level. The Compendium defines L4, L5 and a floor at L3; the names and wording of L1 to L3, and the name of L5, are PragmaWorks working definitions that the Compendium does not yet carry.

- **L1 Ad-hoc**: the practices exist only as intentions; nothing is enforced and the understanding lives in people's heads.
- **L2 Documented**: the artifacts exist, but nothing enforces them, so they drift or get skipped under deadline.
- **L3 Enforced-with-gaps**: automated checks run and block merges, with known holes or manual ways around them.
- **L4 Enforced-and-bound**: no silent way to skip a check, and the artifacts are tied to reality.
- **L5 Self-improving**: a trend across snapshots. The ratchet blocks regressions and quality trends up over at least three dated snapshots (a provisional convention, not an established number).

What a "governed as of a date" claim requires is under the six definitions below.

What the rubric does not measure — specify it anyway. Architectural correctness and supply-chain safety are orthogonal. A repo can score full marks on all seven properties — perfect layers, full enforcement, complete audit trail — and still ship high-severity CVEs pulled in by an unconstrained dependency chain. The rubric grades how the AI *structured* what it produced, not what it *selected*. So state selection as a constraint too: `npm audit` zero-HIGH as a P1 gate, plus an approved/forbidden library list. An executor handed no dependency policy is unconstrained in the supply-chain dimension — and will act like it.

When is your spec complete?

Completeness is not length — it is **closing the output space to the correct programs**. A spec is complete when a stateless reader can derive the *right* thing from it alone; every open degree of freedom is a place it will guess (a descriptive spec floors the model to the literal minimum; in one small test (RND-1, Section 3) a prescriptive one closed what the descriptive one left open). **Assume nothing, but do not specify the how.**

A complete spec MUST carry: identity and boundary (what it is and is not); per feature, **normative acceptance criteria** (RFC 2119 MUST/SHOULD/MAY — every MUST is a probe); closed decisions with their *why* (ADR/EDR); contracts (types, tests-as-spec); constraints and prohibitions; the sentinel routing; and the verifying gates. It MUST NOT carry: the implementation procedure (the executor derives it), over-marking (keyword only the load-bearing lines), or restated defaults the domain schema already implies — over-specification is the same excess that degrades any bounded artifact; match the *specificity dial* to the stakes.

The self-test, per requirement: if you handed only this to a stranger with no context, would they build the right thing or guess? Where they would guess is the missing constraint — add it (reframe the defect as the specification-query that would have ruled it out). Full criterion, an audit-and-complete prompt, and an F-NNN template: genspec.dev/method/spec-completeness/.

Six pathologies you'll recognize, and the property linked to each

The rubric grades structure; the pathologies name what its absence *feels like* in a real repo. These are six entries from the diagnostic catalog (pragmaworks.dev/diagnose), which names more. The property shown is the one that the catalog or the rubric's scoring guide links to the pathology. Only the catalog links Session Amnesia, and only the scoring guide links Specification Absence and Implicit Contract Syndrome; the other three appear in both. Where the catalog lists a pathology under more than one property, only one is shown:

Pathology	What you'll recognize	Property linked to it
Session Amnesia	the model repeats a decision you already corrected	**Self-describing**: the sentinel and its corrections log carry the decision into the next session
Implicit Contract Syndrome	two systems agreed on nothing written (the Mars Climate Orbiter problem)	**Composable**: explicit contracts that both sides derive from
Specification Absence	no sentinel, no architecture decision on record	**Self-describing**: intent and scope are written down, so a reader does not have to infer them
Implicit Architecture	the structure lives in someone's head	**Self-describing**: screaming architecture, where naming and layering announce the structure, so it does not have to be inferred
AI Security Blindspot	the AI runs irreversible operations with no confirmation step	**Defended**: consequences are classified and irreversible actions need a person to confirm. Dependency choice is a separate constraint (see above: `npm audit` gate + approved-library list)
Test Theater	high line coverage, low mutation score	**Verifiable** — mutation measures what was *caught*, not what ran

The loop: retrieve → generate → verify

Author the structure so the agent **retrieves** context instead of re-deriving it; the agent **generates**; the harness **verifies** — the full test pyramid (unit, integration, E2E, mutation, contract, SLO meaning service-level targets such as response time) plus AI-as-QA run against the *live* application, not assumed from a clean compile. The verify step is **generative execution**: for each use case, the agent operates the real system and checks several separate signals (persisted state, the interface, the contract, the logs) against each other and against the specification (the method as described at pragmaworks.dev/harden, not a result). It looks at the result through several channels at once, not only a test run.

Terminology: guides and sensors. Böckeler (2026) divides an agent's harness into *guides* (feedforward) and *sensors* (feedback), each computational or inferential. Here the sentinel, specifications, instruction files and skills are guides; tests, linters, structural checks and gates are sensors, and "harness" used for the verification layer means the sensors. An instruction file is advisory, a hook is deterministic; both layers should derive from the same ratified specification, with the guides kept minimal. Böckeler, B. (2026). Harness engineering for coding agent users. martinfowler.com. https://martinfowler.com/articles/harness-engineering.html

Why the verify step insists on mutation testing. An AI that writes its own tests *knowing the implementation* will write them to pass, not to catch. Line coverage rewards exactly that: a suite that executes every line but asserts nothing scores 100% coverage and 0% mutation score. In the original AX run, a Stryker check (a mutation-testing tool: it changes the code on purpose and counts how many changes the tests notice) on the services layer of the GS-structured project scored **58.6%** on the model's original tests and **93.1%** after two rounds of added tests (demonstration without comparison, exploratory; one project, checked after the experiment and after compile errors were fixed so the tool could run; the added tests were written by the model and repaired until the score rose, so this shows

that mutation testing detects weak tests, not that GS caused better ones). Run mutation *right after each test batch*, not at release; the surviving mutants are the assertions you forgot to write. Coverage measures what was executed; mutation measures what was *caught*.

Every defect becomes a permanent test and a named rule. This is **the ratchet**: the rule set only grows, and each fixed bug adds a check meant to catch that class of failure again. A defect is not "the method failed"; it is a **specification query**: *what constraint, had it been written, would have ruled this out?*

Six definitions, so the claims mean something checkable. They are design, not results. The first two are below; lifecycle coverage, the triage, the snapshot and completeness follow.

- *No new debt per change.* For a change *c* and each tool-measured measure *i* (duplicated lines in the diff, complexity of touched functions, unused exports, layer-rule violations, touched lines without coverage): `delta_i(c) = m_i(after) - m_i(before)`, and the change is admitted only if `delta_i <= 0` for every blocking measure. Advisory measures are reported and do not block until their false-positive rate is measured and near zero. The baseline is stored and the executor cannot edit it. It does **not** mean zero debt overall: it means a change may not make the measured debt worse, and only a run against the stored baseline counts, not a sentence in a status file.

- *Criteria coverage.* `coverage = criteria with a verification method and a passing check / all ratified criteria`. A criterion with no way to verify it counts as uncovered. A person in the product or business role ratifies the criteria (approves them in writing, so they are the agreed definition of done); the executor cannot edit them or the gates.

- *Lifecycle coverage.* Not covered yet: running the system day to day, and retiring it. Post-mortem and auditability are partial. The table with evidence tiers is in the Compendium.

- *Triage of a failure.* When something fails, say which case it is before acting: does the ratified spec already require the right behavior? If yes, a regression test seen failing first is enough and the spec does not change. If the spec was silent, state the gap as a criterion, have a person ratify it, and derive the test from it. If the spec itself was wrong, record the change and ratify it again. A missing tool is added and named in the sentinel. A newly found way around a gate becomes a permanent test case whose count never goes down. Local hooks can be skipped, and only a check on the shared branch enforces. It was checked on one sample project and one real agent run; it is not a measured effect.
- *Governed as of.* A project is never finished, so a score is a snapshot: say which commit and which spec version it was measured against. "Governed as of a date" means each of the seven properties was at level L4 (enforced with no silent skip path, tied to reality) at that snapshot, none below L3, with the evidence in the audit trail, and nothing about later commits. The free audit's letters do not make that claim: they are a first reading by a model, and "governed" needs the evidence behind each property. A team that aims lower reports "L3 declared", never "governed". L5 is a trend across several snapshots, not a strictly monotonic curve.
- *Spec completeness.* Keep three numbers apart: criteria with a passing check, open questions, and places a stranger would guess. Each place goes back into the spec, and a person ratifies it. The numbers count what is written, not what nobody has thought of yet.

Coherence between spec and code. Divergence is a verification problem, not a generation problem: it does not matter how the code was generated, it matters to detect that code and document stopped saying the same. The mechanism has to live in files and on the shared branch, because the session forgets. Five checks, none using a model: identifiers in both directions (every criterion has a test, and every test cites a live criterion, so an orphan test that still passes is found); the lock (code and derived artifacts carry only a stable tag naming the spec section they came from; the hashes of the sections live in one file, like a lockfile, so a reworded rule makes the old test stale until it is regenerated or re-ratified with a reason on record); a co-change gate (a behavior change cites a criterion, and a refactor must pass its parent's tests unchanged); an inverse inventory (every public route or export is claimed by the spec); and an intent diff in the review (criteria touched, stale artifacts, criteria with no test). It detects that spec and code stopped coinciding, not that the spec is wrong and not a test that proves nothing. Status: design; the checks were verified on 35 crafted scenarios in one sample project; no effect was measured. Compendium Section 8.20 and genspec.dev/method/coherence/.

Full text, evidence tiers and the lifecycle table: Compendium 8.19 and genspec.dev/method/lifecycle/.

The horizon: what each tier is designed to remove

Why bother building all this structure? Each tier of the obligation cascade is designed to take one kind of work off the person. That is the design aim, not a measured saving: whether any tier saves effort is untested. The status given with each tier is the Compendium's own label.

- **T1, development**: designed to remove writing code and reading or reviewing each generated line; the harness verifies the code against the specification, and a person ratifies the specification. The Compendium labels T1 demonstrated. The evidence anyone can inspect is the AX series (controlled comparison, exploratory) and the formal-tier run (ALX, demonstration without comparison), with no comparison of effort. The Compendium also counts author-built production cases; it says these support no conclusion about whether the method works.
- **T2, staging**: designed to remove touching deployment by hand and validating the staged system by hand; the spec drives CI/CD. The gate checks the mechanical part, and compliance and legal sign-off stay with people. The Compendium's table marks T2 demonstrated on a case this guide cannot cite; the one inspectable result this guide can point to is the deployed sample service in Section 3 (EX), a single run.
- **T3, production**: designed to remove monitoring and diagnosing by hand, with the spec's behavioral contracts run against the live system. This tier is designed, not demonstrated: it is described at design level only, with no measured diagnostic performance.
- **T4, evolution**, which would take over maintaining and extending the living system, is marked demonstrated in a simulation (the Loom colony). **T5, synthesis**, is designed. **T6, meta-telos**, is a research agenda.

Each tier is admissible only when the one before it holds. The full cascade is treated in the Compendium. The aim is not "the AI writes code faster"; it is that specifying correctly becomes the work that stays with the person.

The sharpest move: prescriptive, not descriptive

The most common objection we hear (no count kept) is *"the agent cuts corners."* It is real, and it is a specification problem. Under speed-and-token pressure, a **descriptive** spec ("build a rate limiter") lets the model floor to the literal minimum. A **prescriptive** spec — intent made explicit ("reject the 101st request in a 60s window with HTTP 429, per API key, return `Retry-After`") — closes the output space. What the spec does not close, the agent is free to floor.

Reach for the RFC 2119 keywords — they are the lexical tool for closing a degree of freedom. Phrase each load-bearing obligation with a capitalized normative word: **MUST** closes the freedom outright (a blocking gate), **SHOULD** is defeasible (a warning — deviation needs a recorded reason), **MAY** is ungated (permitted, unchecked). The keyword sets both the obligation and the gate's severity, so "MUST reject the 101st request in a 60-second window per API key with HTTP 429" closes what "the rate limiter should handle bursts" leaves open. Every MUST is an acceptance criterion, which makes it a machine-checkable probe — the keyword is where a prescriptive clause connects to **Verifiable** and the verify loop. Keyword the obligations that carry weight, not every sentence; over-marking is excess too.

The unit of work: a bound prompt, not a task title

A roadmap line like *"build the connection system"* forces the agent to reconstruct scope at execution time — exactly where it invents. Bind every task to a prompt that carries its own references, scope, and acceptance test:

```
## [ID] — [task name]
**Load:** [exact artifacts to read — and what NOT to load]
**Scope:** [what to build — and, explicitly, what NOT to touch]
**Acceptance:**
- [ ] [specific, verifiable criterion]
- [ ] full suite passes
- [ ] exercised at the HTTP/CLI boundary
**Commit:** feat(scope): [description]
```

The load-bearing line is **what NOT to touch**. Facing a failing test, an agent's path of least resistance is to edit the production code until the test passes; a `NOT IN SCOPE: implementation code` line makes that path less likely; only a gate or hook makes it unreachable. It is the prescriptive move applied to a whole session.

---

3. The numbers

What the experiments found, and how far each result goes. Every study below is exploratory: the author designed the method, wrote the specifications, ran the experiments and scored the results, and no result has been replicated externally. *How* each was produced is in the paper and the linked experiments. Each line says how the result was obtained, how many runs it rests on (*n*, where a run is one generation, not a person), and what it covers. *Exploratory* means the design was not fixed with an outside timestamp before the data, so a result can suggest a pattern and does not test it. *Controlled comparison*

means the researcher set the conditions and changed the factor of interest, with no human participants; where a measure can favor one condition by design, the result says so. *Demonstration without comparison* means one run showing that something can be done. *Post-hoc series* means each condition was designed after seeing the earlier results. *Twin comparison* means two versions of the same system that behave the same and differ along one axis, such as how much duplicated and dead code they carry. Most results use a single sample application and one model family. Read them as evidence that a mechanism exists, not as measured effect sizes.

Caveat: the benefits that come from structure helping a weaker model read and write code are expected to depend on model capacity, and to recede as models improve. The Compendium's capacity studies (Section 7.8) bear on this and are exploratory. None of the results listed below shows how the benefit changes with model capacity (the one that compares two models, MX, found both at the ceiling), and none estimates how fast the benefit fades.

- **Structure (AX).** Five runs each of a naive prompt, an expert prompt and a GS-structured specification built the same RealWorld "Conduit" backend. The GS-structured specification separated from the naive prompt on structure and completeness, with no overlap between the five runs on each side: layer-boundary violations (naive median 45, GS median 0), files emitted (naive 21, GS 55) and test files emitted (naive 5, GS 13). Each has Cliff's delta = 1.0 (complete separation: all five runs on one side fall beyond all five on the other) and a Holm-corrected p of 0.024 (the chance of a split this clean arising by luck, adjusted for the three comparisons made on each measure). With five runs per side, 0.008 is the smallest exact p that exists, so this reads as no overlap in five draws, not a precise size. Part of the gap is built in: the layer rule that the violation count checks was stated in the expert prompt and in the GS specification and not in the naive prompts, and the expert and GS prompts asked for tests with each feature while the naive prompts asked for them only in the last prompt. On the 0 to 14 audit score (scored by AI sessions; first session shown) naive and GS did not separate (Holm p = 0.27). The GS-structured specification was not distinguished from a strong expert prompt either: no difference survived correction on audit score, files, test files or layer violations (Holm p = 0.89 for layer violations). So the data separate GS from no structure on layer violations, files and test files (not on the audit score), and do not separate it from a good prompt. GS runs also cost more to generate: about 2.9 times a naive run and 1.4 times an expert-prompt run (median $1.89 per GS run, against $0.65 naive and $1.34 expert). *(controlled comparison, exploratory; 5 runs per condition, one model (Claude Sonnet 4.5), one benchmark, run and scored by the author)* An earlier series scored the naive prompt 3/14 and the final GS version 14/14: one run per condition, the first three fixed before their runs and each later one designed after seeing the gaps in the one before. It shows that each gap could be named and closed, not an effect size. Those scores use a 14-point rubric that is a legacy instrument, retired as a scorecard in favor of letter grades A to F and a task list; it is kept only because the experiments were scored with it. *(post-hoc series, one run per condition)*
- **Retrieval cost (KX).** Giving the agent a routed map of the project used about a third of the tokens per question that letting it search the code with no structure used (78,603 against 233,583), and about four-fifths of what loading every document into its context used (78,603 against 100,237), averaged over 45 questions. On the 8 of those questions about the layer structure (n = 8; 4 trace a request file by file, and the others ask for the order of the layers, where the interfaces and their database code live, which layer may use the database client, and where the parts are wired together), the routed map used more tokens (142,511) than code search with

no structure (about 106,000) or loading every document (about 101,000). On the 8 questions about what a single function does, plain code search was cheaper than the map (62,649 against 78,936). Accuracy was scored but is not claimed, because the answer key comes from the same structure the map reads. *(controlled comparison, exploratory; 45 questions per condition, one model, one system)*

- **Ordered vs. average code (SX).** Making the same change to a codebase carrying an average project's mess (duplication, dead code, mixed patterns) cost **~2.4× the tokens and 3.5× the edits** of the clean version, *even with a navigation map on both*. A map helps the agent *find* things; with the map on both versions the messy one still cost more, and no map removed that cost. The study did not test whether refactoring removes it. This shows a mechanism, not a size of effect. *(twin comparison of two versions of one sample system, exploratory; n = 2 per version, one model, not statistically tested; the figures are for the case with the map on both)*
- **Model choice (MX).** On one public benchmark (the RealWorld "Conduit" backend, 149 acceptance requests checked by a fixed set of HTTP checks, a Hurl suite, that the test harness runs, not the agent), a mid-tier model (Sonnet 4.6) and a stronger model (Opus 4.8) both passed 149 of 149 from the same specification. The mid-tier model used fewer tokens: 43.4k against 54.4k (totals per model run, no input and output split). Cost was not metered, so no cost multiple is claimed. Both models were at the ceiling, no run lacked the specification, and the benchmark is one they have very likely seen in training, so this does not show whether the specification or the model produced the result. *(single-run comparison of two models, exploratory; n = 1 per model, one benchmark)*
- **Formal tier (ALX).** A compiler derived from its own formal specification: the first run passed none of the acceptance tests. The specification, not the code, was then corrected in five phases, with the tests already written and visible, so the record has six scored points (the baseline and one after each phase), and the final recorded run passed all **386** tests of its suite. The correction log counts 410 tests at the start; the committed evidence does not say why the count changed. *(demonstration without comparison, exploratory; one derivation)*
- **Deployed service (EX).** A public sample backend was specified, generated and deployed to a live runtime: **13/13** behavioral probes (HTTP checks against the live service) passed (1,013 assertions, as reported in the experiment README) and all six thresholds held in a 3-minute load test (five latency and error limits, plus reaching 10 simulated users). This is a small load test of a sample application, not a customer's production system. *(demonstration without comparison, exploratory; one run, one system)*
- **Corner-cutting (RND-1).** A descriptive spec scored **0 of 3** against a check the model never saw; the same task with a prescriptive spec scored **3 of 3**, at about equal token cost (17,749 against 17,508 on the first runs). The test harness ran the check, not the model. The result is close to true by definition, because the prescriptive spec states what the check tests. *(comparison of two versions of one specification, exploratory; 3 runs per version, one task, one model, not statistically tested)* RND-1 also tested bounded context (n = 2) and test-faking (n = 2); at 12 requirements the bounded-context version did no better than the flat one (both 100%), and in the test-faking check the model did not fake its tests, so verification had nothing to catch. The Compendium reports both (Section 7.8.I).

Not claimed. There is no measured end-to-end "your token bill drops X%"; the binding metric is *tokens-per-correct-output*, not tokens spent. Four comparisons above report what generating or completing the work cost: Structure (dollars per run), Ordered vs. average code (tokens to complete a change), Model choice (tokens per model run) and Corner-cutting (tokens on the first runs). Only Structure priced generating from the full specification against plain prompts, and there the full specification cost more per run than either prompt (about 1.4 times the expert prompt and 2.9 times the naive prompt), so absolute spend can go up. Every result above comes from the author's own runs and none has been replicated outside the author's team, so treat each one as a lead to test on your own code, not as settled. Other null results are in the Compendium, such as a twin study in which disciplined layering alone did not make code cheaper to read (TX, Section 7.8.J).

The other side, measured by someone else. A separate study by other authors shows what happens *without* authored structure. Orlanski et al. (2026, arXiv:2603.24755, May 2026 revision) built SlopCodeBench, in which 15 coding agents repeatedly extend their own earlier solutions across 36 problems. No agent fully solved any problem end to end. Structural erosion (complexity piling up in a few already-complex functions) rose in 77% of trajectories (an agent's run of extensions on one problem) and verbosity (redundant or duplicated code) in 75.5%. Compared with 473 open-source Python repositories, the agents' code was 2.3 times more verbose and 2.0 times more eroded, and the human repositories degraded less often and by smaller margins. In the authors' test of a quality-guidance prompt on three GPT agents, it lowered the starting level of verbosity and erosion and, in the authors' words, had little impact on the iterative degradation (for one of the three agents, GPT 5.4, quality did improve from start to end). The researchers did not test GS. This is the failure GS is built to prevent, not evidence that GS prevents it.

---

4. Start here — this week

1. **Create the entry file** before the first agent session: `CLAUDE.md`, `AGENTS.md` or the equivalent file your tool reads, at the repo root, with the five categories. Write down identity, layers and their ownership, the schema, and a skeleton decision record. One door.
2. **Turn on the harness**: hooks, plus a check on the shared branch, that gate on tests, types, and lint. "Done" = gates pass. Local hooks can be skipped, so only the check on the shared branch enforces. You do not need a special tool: give your assistant the gates page (genspec.dev/method/gates/) and ask it to wire the gates that fit your stack. If you want an installer and a checker, genspec-tools is public (MIT license, plain Node 18 or later, and git): `npx github:pragma-works/genspec-tools demo --sample` shows what it looks at without touching your project. It is early: it was tested on example projects written by its author, and nobody else has checked it yet. Its check command runs a project's own install steps, hooks and tests, so run it only on code you trust.
3. **Grade yourself** on the seven-property rubric. Paste the free audit prompt from pragmaworks.dev/audit into your assistant, pointed at your repository. No install. It returns a letter for each of the seven properties and what to fix first. The letters are a first reading by a model: they are not a maturity level, and they do not say the project is governed. Your two lowest letters are your next two moves.

4. **Two more public tools, both optional.** CodeSeeker (Apache-2.0, `npm install -g codeseeker`) is a code-search and code-graph tool for assistants. No result in this guide tests it. Chronicle (`chronicle-mcp`) keeps memory across sessions for an assistant. pragmaworks.dev lists it as in development; its license is PolyForm Small Business (free for companies with fewer than 100 people and under USD 1M a year in revenue; larger ones need a commercial license), and team features need a token.

Hooks aren't just safety, they're budget. By our estimate, not a measurement, a check that runs as a hook adds little to the context, while the same check done in-conversation (compile, run tests, scan for forbidden patterns) costs tokens *each time the agent redoes it by hand*. Move verification into hooks and the freed budget can go to work instead of re-checking. And **Defended** is not earned until hooks actually run: *"add pre-commit hooks"* written in a status file is not a defended system; hooks logging real violations are.

What GS does not remove: the judgment layer. Naming what stays human keeps the claims here honest. GS is designed to automate specification, generation, and verification; it does not touch *domain expertise*, the *strategic decision about what should exist*, real *user research*, *aesthetic judgment*, or *compliance sign-off*. Those are the terminus every tier routes toward, not the work the harness absorbs. If a pitch claims the machine decides what to build, it is overselling; GS is designed to move the work downstream of that decision onto files and checks. Whether that lowers cost is untested, and Section 3 shows generation can cost more.

The specification is the mold. The AI is the foundry. In our view, the scarce resource, the one that does not regenerate for free, is the judgment to specify correctly.

Links

- **White paper** (the full argument and citations): `github.com/jghiringhelli/generative-specification` → `docs/white-paper/GenerativeSpecification_WhitePaper.md`
- **Compendium** (complete evidence, the pathology catalog, the formal treatment): same repo, `docs/white-paper/GenerativeSpecification_Compendium.md`
- **Experiments** (committed run data and write-ups: AX, EX, KX, MX, RND-1, SX): `github.com/jghiringhelli/generative-specification/tree/main/experiments`
- **Formal-tier experiment (ALX)**: `github.com/jghiringhelli/loom/tree/main/experiments/alx`
- **The author's firm** (PragmaWorks offers paid assessments of this kind): `pragmaworks.dev`